Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 4
дисциплины «Искусственный Интеллект и Машинное Обучение»

Выполнил: Мендеш Пашкоал Педру
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная техника», направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета цифровых, робототехнических систем и электроники института перспективной инженерии.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель работы: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Этапы выполнения лабораторных работ

1. Изучение теоретического материала
2. Создание репозитория на GitHub: https://github.com/Pascoalpm/Laboratori-pandas-series.git

Comentado [PM1]: https://github.com/Pascoalpm/Laboratori-pandas-series.git

3. Клонирование репозитория:
4. Настройка .gitignore для Python, Jupyter.
5. Запуск JupyterLab:
6. Выполнение заданий

Примеры практической работы

Пример 1 - Создание Series из списка

```
import pandas as pd

# Пример из теории
s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

Рисунок 1

Пример 2 - Создание Series с пользовательскими индексами

```
import pandas as pd

# Пример из теории
s2 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
print(s2)
```

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Рисунок 2

Пример 3 - Создание Series из массива NumPy

```
import pandas as pd
import numpy as np

# Пример из теории
ndarr = np.array([1, 2, 3, 4, 5])
print(type(ndarr))

s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])
print(s3)
```

```
<class 'numpy.ndarray'>
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Рисунок 3

Пример 4 - Создание Series из словаря

```
import pandas as pd

# Пример из теории
d = {'a': 1, 'b': 2, 'c': 3}
s4 = pd.Series(d)
print(s4)
```

```
a    1
b    2
c    3
dtype: int64
```

Рисунок 4

Пример 5 - Создание Series из константы

```
import pandas as pd

# Пример из теории
a = 7
s5 = pd.Series(a, ['a', 'b', 'c'])
print(s5)
```

```
a    7
b    7
c    7
dtype: int64
```

Рисунок 5

Пример 6 - Доступ к элементам по индексу

```python
import pandas as pd

# Пример из теории
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s6)

# ПРАВИЛЬНЫЙ доступ в pandas:
print("\n1. Доступ по ПОЗИЦИИ (числовой индекс) с помощью .iloc[]:")
print(f"s6.iloc[2] = {s6.iloc[2]}")   # третий элемент

print("\n2. Доступ по МЕТКЕ (индекс 'd') с помощью .loc[]:")
print(f"s6.loc['d'] = {s6.loc['d']}")

print("\n3. Доступ по МЕТКЕ (короткая форма):")
print(f"s6['d'] = {s6['d']}")
```

```
Исходная Series:
a    1
b    2
c    3
d    4
e    5
dtype: int64

1. Доступ по ПОЗИЦИИ (числовой индекс) с помощью .iloc[]:
s6.iloc[2] = 3

2. Доступ по МЕТКЕ (индекс 'd') с помощью .loc[]:
s6.loc['d'] = 4

3. Доступ по МЕТКЕ (короткая форма):
s6['d'] = 4
```

Рисунок 6

Пример 7 - Срезы (slices) в Series

```python
import pandas as pd

# Пример из теории
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s6)

print("\nСрез s6[:2]:")
print(s6[:2])
```

```
Исходная Series:
a    1
b    2
c    3
d    4
e    5
dtype: int64

Срез s6[:2]:
a    1
b    2
dtype: int64
```

Рисунок 7

Пример 8 - Логическая индексация

```python
import pandas as pd

# Пример из теории
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s6)

print("\nЭлементы <= 3 (s6[s6 <= 3]):")
print(s6[s6 <= 3])
```

```
Исходная Series:
a    1
b    2
c    3
d    4
e    5
dtype: int64

Элементы <= 3 (s6[s6 <= 3]):
a    1
b    2
c    3
dtype: int64
```

Рисунок 8

Пример 9 - Векторизированные операции

```python
import pandas as pd

# Пример из теории
s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
s7 = pd.Series([10, 20, 30, 40, 50], ['a', 'b', 'c', 'd', 'e'])

print("s6:")
print(s6)
print("\ns7:")
print(s7)

print("\ns6 + s7:")
print(s6 + s7)

print("\ns6 * 3:")
print(s6 * 3)
```

```
s6:
a    1
b    2
c    3
d    4
e    5
dtype: int64

s7:
a    10
b    20
c    30
d    40
e    50
dtype: int64

s6 + s7:
a    11
b    22
c    33
d    44
e    55
dtype: int64

s6 * 3:
a     3
b     6
c     9
d    12
e    15
dtype: int64
```

Рисунок 9

Пример 10 - Методы .iloc[] и .loc[]

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\n.iloc[0]:", s.iloc[0])
print(".iloc[2]:", s.iloc[2])
print(".iloc[-1]:", s.iloc[-1])

print("\n.loc['a']:", s.loc['a'])
print(".loc['c']:", s.loc['c'])
print(".loc['e']:", s.loc['e'])
```

```
Исходная Series:
a    10
b    20
c    30
d    40
e    50
dtype: int64

.iloc[0]: 10
.iloc[2]: 30
.iloc[-1]: 50

.loc['a']: 10
.loc['c']: 30
.loc['e']: 50
```

Рисунок 10

Пример 11 - Разница в срезах .iloc vs .loc

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\n.iloc[1:3] (индекс 3 не включается):")
print(s.iloc[1:3])

print("\n.loc['b':'d'] (индекс 'd' включается):")
print(s.loc['b':'d'])
```

```
Исходная Series:
a    10
b    20
c    30
d    40
e    50
dtype: int64

.iloc[1:3] (индекс 3 не включается):
b    20
c    30
dtype: int64

.loc['b':'d'] (индекс 'd' включается):
b    20
c    30
d    40
dtype: int64
```

Рисунок 11

Пример 12 - Логическая индексация для фильтрации

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\nФильтрация значений больше 10 (s[s > 10]):")
filtered_s = s[s > 10]
print(filtered_s)
```

```
Исходная Series:
a    10
b    25
c     8
d    30
e    15
dtype: int64

Фильтрация значений больше 10 (s[s > 10]):
b    25
d    30
e    15
dtype: int64
```

Рисунок 12

Пример 13 - Создание булевого массива

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\nБулевый массив (s > 10):")
print(s > 10)
```

```
Исходная Series:
a    10
b    25
c     8
d    30
e    15
dtype: int64

Булевый массив (s > 10):
a    False
b     True
c    False
d     True
e     True
dtype: bool
```

Рисунок 13

Пример 14 - Комбинированные условия

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\nФильтрация значений в диапазоне от 10 до 30 (s[(s >= 10) & (s <= 30)]):")
filtered_s = s[(s >= 10) & (s <= 30)]
print(filtered_s)
```

```
Исходная Series:
a    10
b    25
c     8
d    30
e    15
dtype: int64

Фильтрация значений в диапазоне от 10 до 30 (s[(s >= 10) & (s <= 30)]):
a    10
b    25
d    30
e    15
dtype: int64
```

Рисунок 14

Пример 15 - Фильтрация по значению индекса

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 25, 8, 30, 15], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\nВыбор элементов с индексами 'b' и 'd' (s[s.index.isin(['b', 'd'])]):")
filtered_s = s[s.index.isin(['b', 'd'])]
print(filtered_s)
```

```
Исходная Series:
a    10
b    25
c     8
d    30
e    15
dtype: int64

Выбор элементов с индексами 'b' и 'd' (s[s.index.isin(['b', 'd'])]):
b    25
d    30
dtype: int64
```

Рисунок 15

Пример 16 - Работа с пропущенными значениями .isnull() и .notnull()

```python
import pandas as pd
import numpy as np

# Пример из теории
s_with_nan = pd.Series([10, None, 8, 30, None], index=['a', 'b', 'c', 'd', 'e'])

print("Series с пропущенными значениями:")
print(s_with_nan)

print("\n.isnull() - обнаружение пропущенных значений:")
print(s_with_nan.isnull())

print("\n.notnull() - обнаружение непустых значений:")
print(s_with_nan.notnull())

print("\nФильтрация только непустых значений (s_with_nan[s_with_nan.notnull()]):")
filtered_s = s_with_nan[s_with_nan.notnull()]
print(filtered_s)
```

```
Series с пропущенными значениями:
a    10.0
b     NaN
c     8.0
d    30.0
e     NaN
dtype: float64

.isnull() - обнаружение пропущенных значений:
a    False
b     True
c    False
d    False
e     True
dtype: bool

.notnull() - обнаружение непустых значений:
a     True
b    False
c     True
d     True
e    False
dtype: bool

Фильтрация только непустых значений (s_with_nan[s_with_nan.notnull()]):
a    10.0
c     8.0
d    30.0
dtype: float64
```

Рисунок 15

Пример 17 - Изменение значений элементов в Series

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

print("Исходная Series:")
print(s)

print("\nИзменение значения с индексом 'b' (s.loc['b'] = 25):")
s.loc['b'] = 25
print(s)

print("\nИзменение второго элемента (s.iloc[1] = 50):")
s.iloc[1] = 50
print(s)
```

```
Исходная Series:
a    10
b    20
c    30
d    40
dtype: int64

Изменение значения с индексом 'b' (s.loc['b'] = 25):
a    10
b    25
c    30
d    40
dtype: int64

Изменение второго элемента (s.iloc[1] = 50):
a    10
b    50
c    30
d    40
dtype: int64
```

Рисунок 16

Пример 18 - Методы .head() и .tail()

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40, 50, 60, 70], index=['a', 'b', 'c', 'd', 'e', 'f', 'g'])

print("Исходная Series:")
print(s)

print("\n.head(3) - первые 3 элемента:")
print(s.head(3))

print("\n.head() - первые 5 элементов (по умолчанию):")
print(s.head())

print("\n.tail(3) - последние 3 элемента:")
print(s.tail(3))

print("\n.tail() - последние 5 элементов (по умолчанию):")
print(s.tail())
```

```
Исходная Series:
a    10
b    20
c    30
d    40
e    50
f    60
g    70
dtype: int64

.head(3) - первые 3 элемента:
a    10
b    20
c    30
dtype: int64

.head() - первые 5 элементов (по умолчанию):
a    10
b    20
c    30
d    40
e    50
dtype: int64

.tail(3) - последние 3 элемента:
e    50
f    60
g    70
dtype: int64

.tail() - последние 5 элементов (по умолчанию):
c    30
d    40
e    50
f    60
g    70
dtype: int64
```

Рисунок 17

Пример 19 - Атрибуты .index и .values

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

print("Исходная Series:")
print(s)

print("\nИндексы (.index):")
print(s.index)

print("\nЗначения (.values):")
print(s.values)

print(f"\nПервый индекс (s.index[0]): {s.index[0]}")
print(f"Последний индекс (s.index[-1]): {s.index[-1]}")

print(f"\nПроверка 'b' в индексах ('b' in s.index): {'b' in s.index}")
print(f"Проверка 'z' в индексах ('z' in s.index): {'z' in s.index}")
```

```
Исходная Series:
a    10
b    20
c    30
d    40
e    50
dtype: int64

Индексы (.index):
Index(['a', 'b', 'c', 'd', 'e'], dtype='str')

Значения (.values):
[10 20 30 40 50]

Первый индекс (s.index[0]): a
Последний индекс (s.index[-1]): e

Проверка 'b' в индексах ('b' in s.index): True
Проверка 'z' в индексах ('z' in s.index): False
```

Рисунок 18

Пример 20 - Атрибут .dtype и приведение типов

```python
import pandas as pd

# Пример из теории
s = pd.Series([10, 20, 30, 40, 50])
print("Тип данных int64:", s.dtype)

s1 = pd.Series([1.5, 2.3, 3.7])
s2 = pd.Series(["apple", "banana", "cherry"])
s3 = pd.Series([True, False, True])

print("\ns1 (float64):", s1.dtype)
print("s2 (object):", s2.dtype)
print("s3 (bool):", s3.dtype)

print("\nПреобразование float в int (.astype(int)):")
s1_int = s1.astype(int)
print(s1_int)
print("Новый тип:", s1_int.dtype)
```

```
Тип данных int64: int64

s1 (float64): float64
s2 (object): str
s3 (bool): bool

Преобразование float в int (.astype(int)):
0    1
1    2
2    3
dtype: int64
Новый тип: int64
```

Рисунок 19

Практические задания работы

Задание 1 - Создание Series из списка

```python
"""
Задание 1: Создание Series из списка

Задача:
Создайте Series из списка чисел [5, 15, 25, 35, 45]
с индексами ['a', 'b', 'c', 'd', 'e'].
Выведите его на экран и определите его тип данных.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём Series из списка с указанными индексами
s1 = pd.Series([5, 15, 25, 35, 45], index=['a', 'b', 'c', 'd', 'e'])

# Выводим Series на экран
print("Созданная Series:")
print(s1)

# Определяем и выводим тип данных
print(f"\nТип данных Series: {s1.dtype}")
```

```
Созданная Series:
a     5
b    15
c    25
d    35
e    45
dtype: int64

Тип данных Series: int64
```

Рисунок 20

Задание 2 - Получение элемента Series

```python
Задание 2: Получение элемента Series

Задача:
Дан Series с индексами ['A', 'B', 'C', 'D', 'E']
и значениями [12, 24, 36, 48, 60].
Используйте .loc[] для получения элемента с индексом 'C'
и .iloc[] для получения третьего элемента.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём Series
s2 = pd.Series([12, 24, 36, 48, 60], index=['A', 'B', 'C', 'D', 'E'])

# Выводим Series для наглядности
print("Исходная Series:")
print(s2)

# Получаем элемент с индексом 'C' с помощью .loc[]
element_loc = s2.loc['C']
print(f"\nЭлемент с индексом 'C' (s2.loc['C']): {element_loc}")

# Получаем третий элемент (индекс 2) с помощью .iloc[]
element_iloc = s2.iloc[2]
print(f"Третий элемент (s2.iloc[2]): {element_iloc}")
```

```
Исходная Series:
A    12
B    24
C    36
D    48
E    60
dtype: int64

Элемент с индексом 'C' (s2.loc['C']): 36
Третий элемент (s2.iloc[2]): 36
```

Рисунок 21

Задание 3 - Фильтрация данных с помощью логической индексации

```python
"""
Задание 3: Фильтрация данных с помощью логической индексации

Задача:
Создайте Series из массива NumPy np.array([4, 9, 16, 25, 36, 49, 64]).
Выберите только те элементы, которые больше 20, и выведите результат.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Создаём массив NumPy
arr = np.array([4, 9, 16, 25, 36, 49, 64])

# Создаём Series из массива
s3 = pd.Series(arr)

# Выводим исходную Series
print("Исходная Series:")
print(s3)

# Фильтруем элементы, которые больше 20
filtered_s = s3[s3 > 20]

# Выводим отфильтрованные элементы
print("\nЭлементы, большие 20:")
print(filtered_s)
```

```
Исходная Series:
0     4
1     9
2    16
3    25
4    36
5    49
6    64
dtype: int64

Элементы, большие 20:
3    25
4    36
5    49
6    64
dtype: int64
```

Рисунок 22

Задание 4 - Просмотр первых и последних элементов

```python
"""
Задание 4: Просмотр первых и последних элементов

Задача:
Создайте Series, содержащий 50 случайных целых чисел от 1 до 100
(используйте np.random.randint()).
Выведите первые 7 и последние 5 элементов с помощью .head() и .tail().
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Устанавливаем seed для воспроизводимости результатов
np.random.seed(42)

# Создаём Series из 50 случайных чисел от 1 до 100
s4 = pd.Series(np.random.randint(1, 101, size=50))

# Выводим информацию о Series
print(f"Series создана. Количество элементов: {len(s4)}")
print(f"Первые 5 элементов (по умолчанию):\n{s4.head()}")
print(f"\nПервые 7 элементов (.head(7)):\n{s4.head(7)}")
print(f"\nПоследние 5 элементов (.tail(5)):\n{s4.tail(5)}")
```

```
Series создана. Количество элементов: 50
Первые 5 элементов (по умолчанию):
0    52
1    93
2    15
3    72
4    61
dtype: int32

Первые 7 элементов (.head(7)):
0    52
1    93
2    15
3    72
4    61
5    21
6    83
dtype: int32

Последние 5 элементов (.tail(5)):
45    51
46     7
47    21
48    73
49    39
dtype: int32
```

Рисунок 23

Задание 5 - Определение типа данных Series

```python
Задание 5: Определение типа данных Series

Задача:
Создайте Series из списка ['cat', 'dog', 'rabbit', 'parrot', 'fish'].
Определите тип данных с помощью .dtype,
затем преобразуйте его в category с помощью .astype().
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём Series из списка строк
s5 = pd.Series(['cat', 'dog', 'rabbit', 'parrot', 'fish'])

# Выводим исходную Series
print("Исходная Series:")
print(s5)

# Определяем и выводим текущий тип данных
print(f"\nТекущий тип данных (dtype): {s5.dtype}")

# Преобразуем тип данных в 'category'
s5_category = s5.astype('category')

# Выводим результат преобразования
print("\nSeries после преобразования в category:")
print(s5_category)

# Выводим новый тип данных
print(f"\nНовый тип данных (dtype): {s5_category.dtype}")
```

```
Исходная Series:
0       cat
1       dog
2    rabbit
3    parrot
4      fish
dtype: str

Текущий тип данных (dtype): str

Series после преобразования в category:
0       cat
1       dog
2    rabbit
3    parrot
4      fish
dtype: category
Categories (5, str): ['cat', 'dog', 'fish', 'parrot', 'rabbit']

Новый тип данных (dtype): category
```

Рисунок 24

Задание 6 - Проверка пропущенных значений

```python
"""
Задание 6: Проверка пропущенных значений

Задача:
Создайте Series с данными [1.2, np.nan, 3.4, np.nan, 5.6, 6.8].
Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN),
и выведите индексы таких элементов.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Создаём Series с пропущенными значениями
s6 = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])

# Выводим исходную Series
print("Исходная Series с пропущенными значениями:")
print(s6)

# Проверяем, есть ли пропущенные значения
has_nan = s6.isnull().any()
print(f"\nЕсть ли пропущенные значения в Series? {has_nan}")

# Находим индексы элементов, содержащих NaN
nan_indices = s6[s6.isnull()].index.tolist()
print(f"Индексы пропущенных значений (NaN): {nan_indices}")

# Выводим булевую маску для наглядности
print("\nБулевая маска (True - пропущенное значение):")
print(s6.isnull())
```

```
Исходная Series с пропущенными значениями:
0    1.2
1    NaN
2    3.4
3    NaN
4    5.6
5    6.8
dtype: float64

Есть ли пропущенные значения в Series? True
Индексы пропущенных значений (NaN): [1, 3]

Булевая маска (True - пропущенное значение):
0    False
1     True
2    False
3     True
4    False
5    False
dtype: bool
```

Рисунок 25

Задание 7 - Заполнение пропущенных значений

```python
"""
Задание 7: Заполнение пропущенных значений

Задача:
Используйте Series из предыдущего задания
и замените все NaN на среднее значение всех непустых элементов.
Выведите результат.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Создаём Series с пропущенными значениями
s7 = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])

# Выводим исходную Series
print("Исходная Series с пропущенными значениями:")
print(s7)

# Вычисляем среднее значение непустых элементов
mean_value = s7.mean()
print(f"\nСреднее значение непустых элементов: {mean_value}")

# Заменяем NaN на среднее значение
s7_filled = s7.fillna(mean_value)

# Выводим результат
print("\nSeries после замены NaN на среднее значение:")
print(s7_filled)
```

```
Исходная Series с пропущенными значениями:
0    1.2
1    NaN
2    3.4
3    NaN
4    5.6
5    6.8
dtype: float64

Среднее значение непустых элементов: 4.25

Series после замены NaN на среднее значение:
0    1.20
1    4.25
2    3.40
3    4.25
4    5.60
5    6.80
dtype: float64
```

Рисунок 26

Задание 8 - Арифметические операции с Series

```python
"""
Задание 8: Арифметические операции с Series

Задача:
Создайте два Series:
- s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
- s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])

Выполните сложение s1 + s2.
Объясните, почему в результате появляются NaN, и замените их на 0.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём первый Series
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

# Создаём второй Series
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])

# Выводим оба Series
print("Series s1:")
print(s1)
print("\nSeries s2:")
print(s2)

# Выполняем сложение
s_sum = s1 + s2
print("\nРезультат сложения s1 + s2 (с NaN):")
print(s_sum)

# Объяснение появления NaN
print("\nПочему появляются NaN?")
print("Индексы 'a' из s1 и 'e' из s2 не имеют соответствующих пар в другой Series.")
print("Pandas не может выполнить операцию для этих индексов и вставляет NaN.")

# Заменяем NaN на 0 при сложении
s_sum_filled = s1.add(s2, fill_value=0)
print("\nРезультат сложения с заменой NaN на 0 (fill_value=0):")
print(s_sum_filled)
```

```
Series s1:
a    10
b    20
c    30
d    40
dtype: int64

Series s2:
b     5
c    15
d    25
e    35
dtype: int64

Результат сложения s1 + s2 (с NaN):
a     NaN
b    25.0
c    45.0
d    65.0
e     NaN
dtype: float64

Почему появляются NaN?
Индексы 'a' из s1 и 'e' из s2 не имеют соответствующих пар в другой Series.
Pandas не может выполнить операцию для этих индексов и вставляет NaN.

Результат сложения с заменой NaN на 0 (fill_value=0):
a    10.0
b    25.0
c    45.0
d    65.0
e    35.0
dtype: float64
```

Рисунок 27

Задание 9 - Применение функции к Series

```python
"""
Задание 9: Применение функции к Series

Задача:
Создайте Series из чисел [2, 4, 6, 8, 10].
Напишите код, который применяет к каждому элементу
функцию вычисления квадратного корня с помощью .apply(np.sqrt).
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Создаём Series
s9 = pd.Series([2, 4, 6, 8, 10])

# Выводим исходную Series
print("Исходная Series:")
print(s9)

# Применяем функцию квадратного корня к каждому элементу
s9_sqrt = s9.apply(np.sqrt)

# Выводим результат
print("\nКвадратный корень каждого элемента (.apply(np.sqrt)):")
print(s9_sqrt)
```

```
Исходная Series:
0     2
1     4
2     6
3     8
4    10
dtype: int64

Квадратный корень каждого элемента (.apply(np.sqrt)):
0    1.414214
1    2.000000
2    2.449490
3    2.828427
4    3.162278
dtype: float64
```

Рисунок 28

Задание 10 - Основные статистические методы

```python
"""
Задание 10: Основные статистические методы

Задача:
Создайте Series из 20 случайных чисел от 50 до 150
(используйте np.random.randint).
Найдите сумму, среднее, минимальное и максимальное значение.
Выведите также стандартное отклонение.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Устанавливаем seed для воспроизводимости результатов
np.random.seed(42)

# Создаём Series из 20 случайных чисел от 50 до 150
s10 = pd.Series(np.random.randint(50, 151, size=20))

# Выводим исходную Series
print("Series из 20 случайных чисел от 50 до 150:")
print(s10)

# Вычисляем и выводим статистические показатели
print(f"\nСумма всех элементов: {s10.sum()}")
print(f"Среднее значение: {s10.mean():.2f}")
print(f"Минимальное значение: {s10.min()}")
print(f"Максимальное значение: {s10.max()}")
print(f"Стандартное отклонение: {s10.std():.2f}")

# Дополнительно выводим сводную статистику через .describe()
print("\nСводная статистика (.describe()):")
print(s10.describe())
```

```
Series из 20 случайных чисел от 50 до 150:
0     101
1     142
2      64
3     121
4     110
5      70
6     132
7     136
8     124
9     124
10    137
11    149
12     73
13     52
14     71
15    102
16     51
17    137
18     79
19     87
dtype: int32

Сумма всех элементов: 2062
Среднее значение: 103.10
Минимальное значение: 51
Максимальное значение: 149
Стандартное отклонение: 32.29

Сводная статистика (.describe()):
count     20.000000
mean     103.100000
std       32.289643
min       51.000000
25%       72.500000
50%      106.000000
75%      133.000000
max      149.000000
dtype: float64
```

Рисунок 29

Задание 11 - Работа с временными рядами

```python
"""
Задание 11: Работа с временными рядами

Задача:
Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года
(pd.date_range(start='2024-03-01', periods=10, freq='D')),
а значениями — случайные числа от 10 до 100.
Выберите данные за 5–8 марта.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np

# Устанавливаем seed для воспроизводимости результатов
np.random.seed(42)

# Создаём диапазон дат с 1 по 10 марта 2024 года
dates = pd.date_range(start='2024-03-01', periods=10, freq='D')

# Создаём Series со случайными значениями от 10 до 100
s11 = pd.Series(np.random.randint(10, 101, size=10), index=dates)

# Выводим полную Series
print("Временной ряд с 1 по 10 марта 2024 года:")
print(s11)

# Выбираем данные за 5–8 марта
print("\nДанные за 5–8 марта 2024 года:")
print(s11['2024-03-05':'2024-03-08'])
```

```
Временной ряд с 1 по 10 марта 2024 года:
2024-03-01    61
2024-03-02    24
2024-03-03    81
2024-03-04    70
2024-03-05    30
2024-03-06    92
2024-03-07    96
2024-03-08    84
2024-03-09    84
2024-03-10    97
Freq: D, dtype: int32

Данные за 5–8 марта 2024 года:
2024-03-05    30
2024-03-06    92
2024-03-07    96
2024-03-08    84
Freq: D, dtype: int32
```

Рисунок 30

Задание 12 - Проверка уникальности индексов

```python
"""
Задание 12: Проверка уникальности индексов

Задача:
Создайте Series с индексами ['A', 'B', 'A', 'C', 'D', 'B']
и значениями [10, 20, 30, 40, 50, 60].
Проверьте, являются ли индексы уникальными.
Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём Series с повторяющимися индексами
s12 = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C', 'D', 'B'])

# Выводим исходную Series
print("Исходная Series с повторяющимися индексами:")
print(s12)

# Проверяем, являются ли индексы уникальными
print(f"\nИндексы уникальны? {s12.index.is_unique}")

# Группируем повторяющиеся индексы и суммируем значения
s12_grouped = s12.groupby(s12.index).sum()

# Выводим результат группировки
print("\nРезультат группировки индексов (сумма значений для каждого индекса):")
print(s12_grouped)
```

```
Исходная Series с повторяющимися индексами:
A    10
B    20
A    30
C    40
D    50
B    60
dtype: int64

Индексы уникальны? False

Результат группировки индексов (сумма значений для каждого индекса):
A    40
B    80
C    40
D    50
dtype: int64
```

Рисунок 31

Задание 13 - Преобразование строковых дат в DatetimeIndex

```python
"""
Задание 13: Преобразование строковых дат в DatetimeIndex

Задача:
Создайте Series, где индексами будут строки
['2024-03-10', '2024-03-11', '2024-03-12'],
а значениями [100, 200, 300].
Преобразуйте индексы в DatetimeIndex и выведите тип данных индекса.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём Series со строковыми индексами дат
s13 = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '2024-03-12'])

# Выводим исходную Series
print("Исходная Series (индексы - строки):")
print(s13)
print(f"\nТип индекса до преобразования: {type(s13.index)}")

# Преобразуем индексы в DatetimeIndex
s13.index = pd.to_datetime(s13.index)

# Выводим результат
print("\nSeries после преобразования индекса в DatetimeIndex:")
print(s13)
print(f"Тип индекса после преобразования: {type(s13.index)}")
```

```
Исходная Series (индексы - строки):
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64

Тип индекса до преобразования: <class 'pandas.Index'>

Series после преобразования индекса в DatetimeIndex:
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
Тип индекса после преобразования: <class 'pandas.DatetimeIndex'>
```

Рисунок 32

Задание 14 - Чтение данных из CSV-файла

```python
"""
Задание 14: Чтение данных из CSV-файла

Задача:
Создайте CSV-файл data.csv со следующими данными:

Дата,Цена
2024-03-01,100
2024-03-02,110
2024-03-03,105
2024-03-04,120
2024-03-05,115

Прочитайте файл и создайте Series, используя "Дата" в качестве индекса.
"""

# Импортируем библиотеку pandas
import pandas as pd

# Создаём CSV-файл
csv_content = """Дата,Цена
2024-03-01,100
2024-03-02,110
2024-03-03,105
2024-03-04,120
2024-03-05,115
"""

with open('data.csv', 'w', encoding='utf-8') as f:
    f.write(csv_content)

print("CSV-файл 'data.csv' создан.")

# Читаем CSV-файл
df = pd.read_csv('data.csv')

# Создаём Series, используя столбец "Дата" в качестве индекса
s14 = pd.Series(df['Цена'].values, index=pd.to_datetime(df['Дата']))


# Выводим результат
print("\nSeries, созданная из CSV-файла (индекс - Дата):")
print(s14)

# Выводим тип индекса
print(f"\nТип индекса: {type(s14.index)}")
```

```
CSV-файл 'data.csv' создан.

Series, созданная из CSV-файла (индекс - Дата):
Дата
2024-03-01    100
2024-03-02    110
2024-03-03    105
2024-03-04    120
2024-03-05    115
dtype: int64

Тип индекса: <class 'pandas.DatetimeIndex'>
```

Рисунок 32

Задание 15 - Построение графика на основе Series

```python
"""
Задание 15: Построение графика на основе Series

Задача:
Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года,
а значениями — случайные числа от 50 до 150.
Постройте график значений с помощью matplotlib.
Добавьте заголовок, подписи осей и сетку.
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Устанавливаем seed для воспроизводимости результатов
np.random.seed(42)

# Создаём диапазон дат с 1 по 30 марта 2024 года
dates = pd.date_range(start='2024-03-01', periods=30, freq='D')

# Создаём Series со случайными значениями от 50 до 150
s15 = pd.Series(np.random.randint(50, 151, size=30), index=dates)

# Выводим первые 5 элементов для ознакомления
print("Первые 5 элементов временного ряда:")
print(s15.head())

# Создаём график
plt.figure(figsize=(12, 6))
plt.plot(s15.index, s15.values, marker='o', linestyle='-', color='blue', markersize=4)

# Добавляем заголовок и подписи осей
plt.title('Временной ряд за март 2024 года', fontsize=14)
plt.xlabel('Дата', fontsize=12)
plt.ylabel('Значение', fontsize=12)

# Добавляем сетку
plt.grid(True, alpha=0.3)

# Поворачиваем подписи дат для лучшей читаемости
plt.xticks(rotation=45)

# Отображаем график
plt.tight_layout()
plt.savefig('grafico_zadanie15.png', dpi=150)
plt.show()

print("\nГрафик сохранён как 'grafico_zadanie15.png'")
```

```
Первые 5 элементов временного ряда:
2024-03-01    101
2024-03-02    142
2024-03-03     64
2024-03-04    121
2024-03-05    110
Freq: D, dtype: int32
```

Рисунок 33

Рисунок 34

Индивидуальное задание

Задание 17 - Отслеживание выбросов в производственных данных

```python
"""
Задание 17: Отслеживание выбросов в производственных данных

Задача:
Создайте Series, где индексами будут часы суток
(pd.date_range('2024-11-15', periods=24, freq='h')),
а значениями — случайное количество дефектных изделий (от 0 до 10).
Преобразуйте индекс в DatetimeIndex, найдите выбросы (значения выше 8)
и выделите их на графике.

Примечание: В pandas частота для часов указывается как 'h' (маленькая буква),
а не 'H' (большая).
"""

# Импортируем необходимые библиотеки
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Устанавливаем seed для воспроизводимости результатов
np.random.seed(42)

# Создаём диапазон часов для 15 ноября 2024 года
# ВНИМАНИЕ: частота 'h' (маленькая буква), а не 'H'
hours = pd.date_range('2024-11-15', periods=24, freq='h')

# Создаём Series со случайным количеством дефектов от 0 до 10
defects = np.random.randint(0, 11, size=24)

# Добавляем искусственные выбросы (значения выше 8)
defects[3] = 9    # 03:00
defects[7] = 10   # 07:00
defects[14] = 9   # 14:00
defects[21] = 10  # 21:00

# Создаём Series
s17 = pd.Series(defects, index=hours)

# Выводим информацию о Series
print("Series почасовых данных о дефектах за 15.11.2024:")
print(s17)
print(f"\nТип индекса: {type(s17.index)}")

# Находим выбросы (значения > 8)
outliers = s17[s17 > 8]
print(f"\nОбнаруженные выбросы (значения > 8):")
print(outliers)

# Создаём график
plt.figure(figsize=(14, 6))

# График всех значений
plt.plot(s17.index, s17.values, marker='o', linestyle='-',
         color='blue', markersize=6, label='Количество дефектов')

# Выделяем выбросы красным цветом
plt.scatter(outliers.index, outliers.values, color='red',
            s=120, zorder=5, label='Выбросы (>8)')

# Добавляем заголовок и подписи осей
plt.title('Почасовое количество дефектных изделий - 15 ноября 2024 года', fontsize=14)
plt.xlabel('Время (часы)', fontsize=12)
plt.ylabel('Количество дефектов', fontsize=12)

# Добавляем сетку
plt.grid(True, alpha=0.3)

# Поворачиваем подписи времени
plt.xticks(rotation=45)

# Добавляем легенду
plt.legend()

# Отображаем график
plt.tight_layout()
plt.savefig('grafico_zadanie17.png', dpi=150)
plt.show()

print("\nГрафик с выделенными выбросами сохранён как 'grafico_zadanie17.png'")
```

Рисунок 35

```
Series почасовых данных о дефектах за 15.11.2024:
2024-11-15 00:00:00     6
2024-11-15 01:00:00     3
2024-11-15 02:00:00    10
2024-11-15 03:00:00     9
2024-11-15 04:00:00     4
2024-11-15 05:00:00     6
2024-11-15 06:00:00     9
2024-11-15 07:00:00    10
2024-11-15 08:00:00     6
2024-11-15 09:00:00    10
2024-11-15 10:00:00    10
2024-11-15 11:00:00     7
2024-11-15 12:00:00     4
2024-11-15 13:00:00     3
2024-11-15 14:00:00     9
2024-11-15 15:00:00     7
2024-11-15 16:00:00     2
2024-11-15 17:00:00     5
2024-11-15 18:00:00     4
2024-11-15 19:00:00     1
2024-11-15 20:00:00     7
2024-11-15 21:00:00    10
2024-11-15 22:00:00     1
2024-11-15 23:00:00     4
Freq: h, dtype: int32

Тип индекса: <class 'pandas.DatetimeIndex'>

Обнаруженные выбросы (значения > 8):
2024-11-15 02:00:00    10
2024-11-15 03:00:00     9
2024-11-15 06:00:00     9
2024-11-15 07:00:00    10
2024-11-15 09:00:00    10
2024-11-15 10:00:00    10
2024-11-15 14:00:00     9
2024-11-15 21:00:00    10
dtype: int32
```

Рисунок 36

Рисунок 37

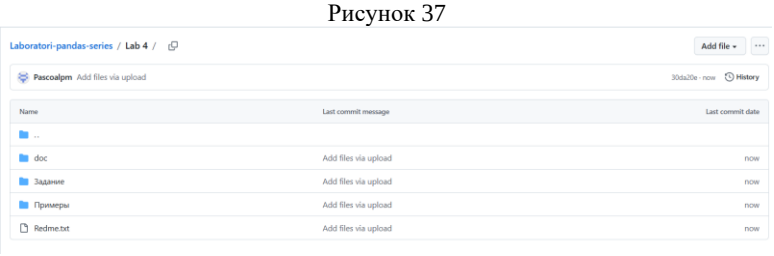

Рисунок 38

ОТВЕТЫ НА ВОПРОСЫ

1. **Что такое pandas.Series и чем она отличается от списка в Python?**

pandas.Series — это маркированная одномерная структура данных. Её можно представить как таблицу с одной строкой. Отличия от списка Python:

- Series имеет индексы (метки) для доступа к элементам
- Можно обращаться как по номеру индекса, так и по метке (как к словарю)
- Поддерживает векторизированные операции
- Имеет встроенные методы для работы с пропущенными данными

2. **Какие типы данных можно использовать для создания Series?**

Для создания Series можно использовать:

- словари Python
- списки Python
- массивы из NumPy (ndarray)
- скалярные величины (константы)

3. **Как задать индексы при создании Series?**

Индексы задаются с помощью параметра index в конструкторе класса:

```
pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
```

4. **Каким образом можно обратиться к элементу Series по его индексу?**

- По числовому индексу: s[2] (как список)
- По метке: s['d'] (как словарь)
- С помощью .loc[] по метке: s.loc['d']

- С помощью .iloc[] по позиции: s.iloc[2]

5. **В чём разница между .iloc[] и .loc[] при индексации Series?**
   - .iloc[] — позиционная индексация (доступ по порядковому номеру, начиная с 0)
   - .loc[] — меточная индексация (доступ по пользовательской метке)

Разница в срезах:

- iloc[start:stop] — крайний индекс не включается (как в стандартных срезах Python)
- loc[start:stop] — крайний индекс включается в выборку

6. **Как использовать логическую индексацию в Series?**

Логическая индексация создаёт булевый массив (True/False) и применяет его к Series:

```
s[s > 10]  # только элементы больше 10
```

Можно использовать комбинированные условия:

```
s[(s >= 10) & (s <= 30)]  # оператор "и"
s[(s < 10) | (s > 30)]    # оператор "или"
```

7. **Какие методы можно использовать для просмотра первых и последних элементов Series?**
   - .head(n) — возвращает первые n элементов (по умолчанию 5)
   - .tail(n) — возвращает последние n элементов (по умолчанию 5)

8. **Как проверить тип данных элементов Series?**

С помощью атрибута .dtype:

```
s.dtype
Пример: int64, float64, object, bool
```

9. **Каким способом можно изменить тип данных Series?**

С помощью метода .astype():

s.astype(int) # преобразование в целые числа

s.astype('category') # преобразование в категориальный тип

10. Как проверить наличие пропущенных значений в Series?

С помощью методов:

.isnull() — возвращает True для пропущенных значений (NaN) # булевая маска

.notnull() — возвращает True для непустых значений # проверка, есть ли хотя бы один NaN

11. Какие методы используются для заполнения пропущенных значений в Series?

- .fillna(value) — заменяет все NaN на указанное значение
- Можно заполнить средним: s.fillna(s.mean())
- Можно заполнить медианой: s.fillna(s.median())

12. Чем отличается метод .fillna() от .dropna()?

- .fillna(value) — заменяет пропущенные значения на указанное значение
- .dropna() — удаляет все элементы, содержащие NaN, и возвращает новую Series без пропущенных значений

13. Какие математические операции можно выполнять с Series?

- Арифметические операции с числом (применяются ко всем элементам): s * 2, s + 10
- Операции между двумя Series (выполняются поэлементно, используя индексы)
- Математические функции из NumPy: np.log(s), np.exp(s), np.sqrt(s), np.sin(s), np.cos(s)

14. В чём преимущество векторизированных операций по сравнению с циклами Python?

- Операции выполняются быстрее, так как реализованы на низкоуровневых языках (C)
- Код становится более компактным и читаемым
- Не циклы for
- Автоматическое выравнивание данных по индексам

15. Как применить пользовательскую функцию к каждому элементу Series?

С помощью метода .apply(func):

```
s.apply(lambda x: x ** 2)          # анонимная функция
s.apply(np.sqrt)                   # функция из NumPy
s.apply(пользовательская_функция)  # пользовательская функция
```

16. Какие агрегирующие функции доступны в Series?

- .sum() — сумма элементов
- .mean() — среднее арифметическое
- .min() — минимальное значение
- .max() — максимальное значение
- .std() — стандартное отклонение
- .describe() — сводная статистика (количество, среднее, стандартное отклонение, минимум, квартили, максимум)

17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series?

```
s.min()   # минимальное значение
s.max()   # максимальное значение
s.mean()  # среднее значение
s.std()   # стандартное отклонение
```

18. Как сортировать Series по значениям и по индексам?

- По значениям: .sort_values()
- По индексам: .sort_index()

Параметр ascending=False для сортировки по убыванию:

```
s.sort_values(ascending=False)
s.sort_index(ascending=False)
```

19.Как проверить, являются ли индексы Series уникальными?

С помощью атрибута .is_unique:

```
s.index.is_unique   # возвращает True или False
```

20. Как сбросить индексы Series и сделать их числовыми?

С помощью метода .reset_index():

```
s.reset_index(drop=True)   # удаляет старый индекс
s.reset_index()            # превращает старый индекс в столбец
```

21. Как можно задать новый индекс в Series?

Прямым присваиванием нового списка индексов:

```
s.index = ['x', 'y', 'z', 'w']
```

22. Как работать с временными рядами в Series?

Временные ряды строятся с использованием индекса типа DatetimeIndex:

```
dates = pd.date_range(start='2024-03-01', periods=5, freq='D')
s = pd.Series([100, 105, 102, 98, 110], index=dates)
```

Можно задавать разную частоту: 'D' (дни), 'H' (часы), 'T' (минуты)

23. Как преобразовать строковые даты в формат DatetimeIndex?

С помощью функции pd.to_datetime():

```
s.index = pd.to_datetime(s.index)
```

24. Каким образом можно выбрать данные за определённый временной диапазон?

```
s['2024-03-02':'2024-03-04']   # обе границы включаются
```

25. Как загрузить данные из CSV-файла в Series?

```
df = pd.read_csv("data.csv")          # читаем DataFrame
```

```
s = df["Цена"]                    # извлекаем столбец в Series
```

Или сразу с указанием индекса:

```
s = pd.read_csv("data.csv", index_col="Дата")["Цена"]
```

26. Как установить один из столбцов CSV-файла в качестве индекса Series?

С помощью параметра index_col:

```
df = pd.read_csv("data.csv", index_col="Дата")
s = df["Цена"]
```

27. Для чего используется метод .rolling().mean() в Series?

Для вычисления скользящего среднего (moving average). Метод создаёт окно фиксированной длины, внутри которого вычисляется среднее значение. Используется для сглаживания данных и анализа трендов.

```
s.rolling(window=3).mean()
```

28. Как работает метод .pct_change()? Какие задачи он решает?

Вычисляет процентное изменение между текущим и предыдущим значением по формуле:

```
pct_change = (текущее - предыдущее) / предыдущее × 100%
```

Для первого элемента результат будет NaN. Используется в финансовом анализе для расчёта доходности акций, валют и других временных рядов.

29. В каких ситуациях полезно использовать .rolling() и .pct_change()?

- Анализ финансовых данных (изменение курса валют, цен на акции)
- Прогнозирование продаж
- Анализ температуры и погодных данных
- Выявление трендов и сглаживание краткосрочных колебаний

- Расчёт волатильности и доходности активов

30. Почему NaN могут появляться в Series, и как с ними работать?

Причины появления NaN:

- Несовпадающие индексы при операциях между Series
- Чтение данных из файлов с пропусками
- Деление на ноль
- Логарифм отрицательного числа или нуля

Методы работы с NaN:

- .isnull() — обнаружение пропущенных значений
- .notnull() — обнаружение непустых значений
- .fillna(value) — замена NaN на указанное значение
- .dropna() — удаление элементов с NaN
- Агрегирующие функции (.sum(), .mean()) игнорируют NaN по умолчанию